

News Classification Based on LLMs

- Project 12

Background

Background

- 10% increase in classification accuracy boosts ad click-through rates by 3-5%.
- While increasing paywall conversion rates for categorized financial/technology content by 27%.
- Our system demonstrates particularly strong commercial potential.
- Reducing manual review costs while maintaining high throughput of 100,000 articles processed daily at just 1.2 seconds per article.



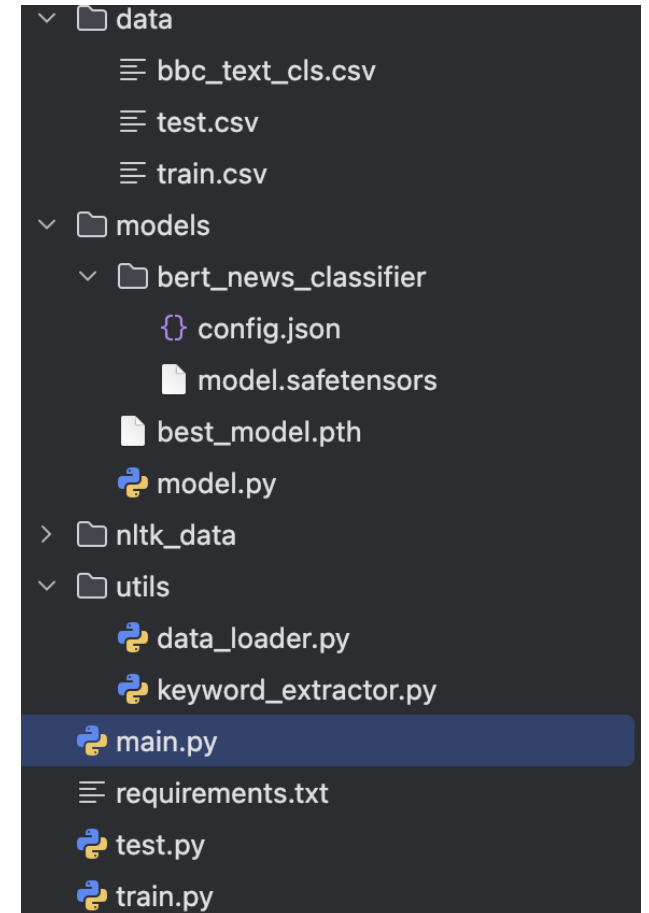
Project Overview

- Developed an LLM chatbot for automated news text processing and classification.
- Category classification (Business, Politics, Entertainment, Sport, Technology) using context-aware keyword extraction.
- Utilized pre-trained DistilBERT model for text processing, considering computational power and dataset size.
- Enhanced model performance through Fine-tuning, Retrieval-Augmented Generation (RAG), and Prompt Engineering

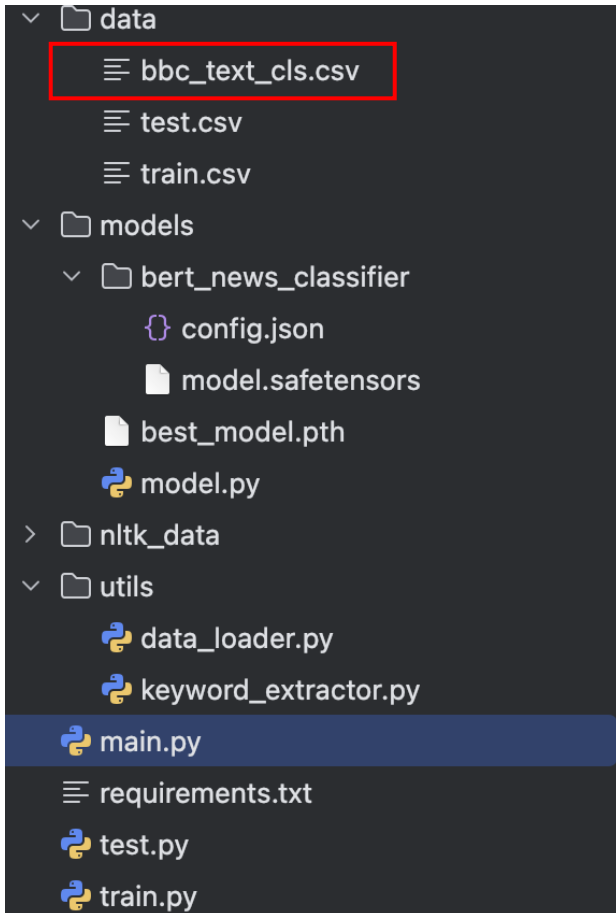
Fine-Tuning Engineering -Based DistilBERT

Fine-Tuning Engineering Procedures

1. Dataset Preprocessing
2. Partition dataset into batches
3. Load the Pre-trained model
4. Setup additional parameters if PEFT is used
5. Determine training arguments
6. Train the model
7. Save the trained model



Dataset Preprocessing



```
      text  labels
0  Ad sales boost Time Warner profit\n\nQuarterly...  business
1  Dollar gains on Greenspan speech\n\nThe dollar...  business
2  Yukos unit buyer faces loan claim\n\nThe owner...  business
3  High fuel prices hit BA's profits\n\nBritish A...  business
4  Pernod takeover talk lifts Domecq\n\nShares in...  business
```

df.head()

```
['business' 'entertainment' 'politics' 'sport' 'tech']
```

labels.unique

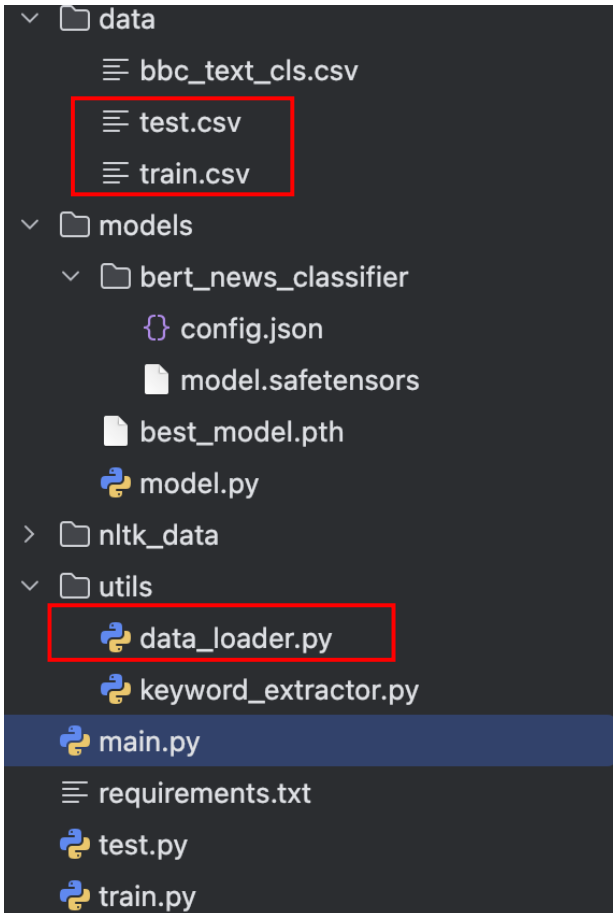
```
RangeIndex: 2225 entries, 0 to 2224
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    text    2225 non-null   object
1   labels  2225 non-null   object
dtypes: object(2)
memory usage: 34.9+ KB
None
```

df.info()

Original dataset includes:

text, labels, no_sentences, Flesch Reading Ease Score, Dale-Chall Readability Score, text_rank_summary, lsa_summary

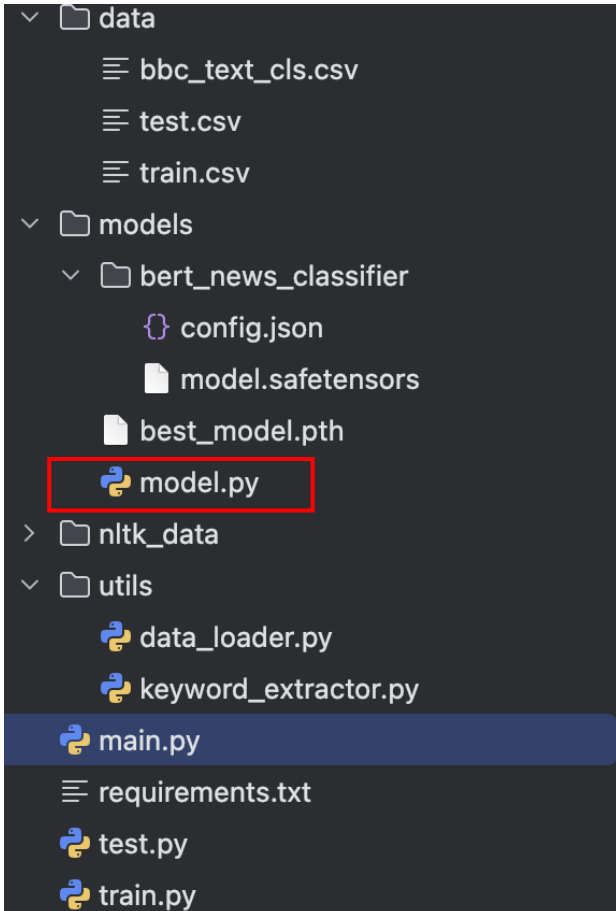
Partition dataset



```
test_size=0.2]
```

```
training dataset: 1780  
testing dataset: 445
```

Load Pre-trained Model



```
import torch.nn as nn
from transformers import DistilBertModel, DistilBertTokenizer
import nltk
```

distilbert/distilbert-base-uncased

Model size 67M params

A distilled and light-weight version of the BERT base model.

- torch provides the core functionality of PyTorch for building and training neural networks.
- DistilBertModel is a lightweight BERT model for extracting text features.
- DistilBertTokenizer is used to convert text into an input format acceptable to the model.
- nltk is a natural language processing toolkit.

Load Pre-trained Model

```
class NewsClassifier(nn.Module): 4 用法
    def __init__(self, num_classes):
        super().__init__()
        self.bert = DistilBertModel.from_pretrained('distilbert-base-uncased')
        self.classifier = nn.Linear(in_features=768, num_classes=num_classes)

    def forward(self, input_ids, attention_mask):
        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
        return self.classifier(outputs.last_hidden_state[:, 0, :])

class TextProcessor: 4 用法
    def __init__(self, max_length=128):
        self.tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')
        self.max_length = max_length

    def encode_text(self, text): 2 个用法 (1 个动态)
        return self.tokenizer.encode_plus(
            text,
            max_length=self.max_length,
            padding='max_length',
            truncation=True,
            return_tensors='pt'
        )
```

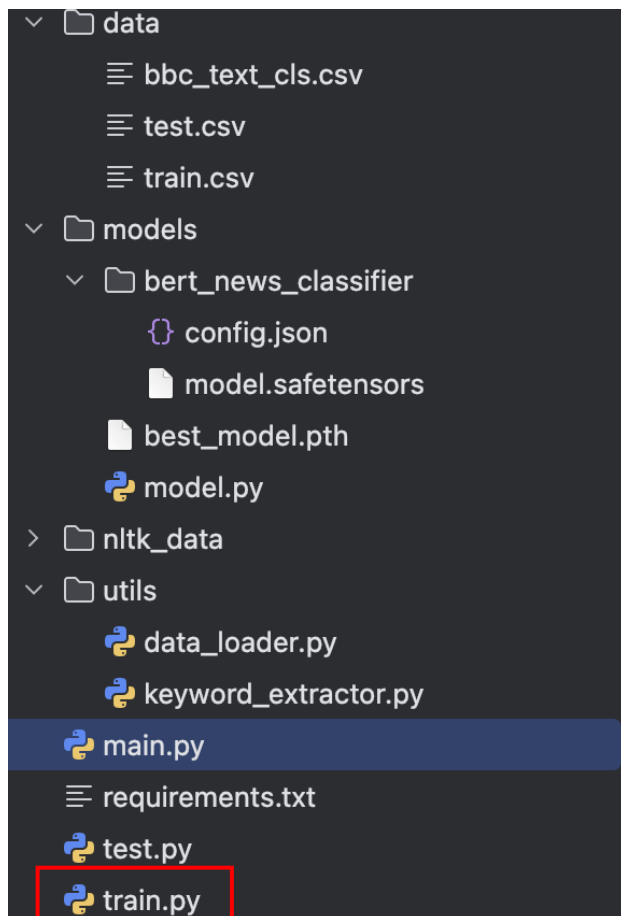
NewsClassifier:

- A news classification model was constructed based on DistilBertModel.
- Uses a fully connected layer to map BERT extracted features to a specified number of categories.

TextProcessor:

- Provides text preprocessing to convert raw text into an input format that the model can handle.
- Uses the DistilBertTokenizer for tokenization and sets maximum length, padding and truncation rules.

Train the Model & Set Arguments



```
class NewsDataset(Dataset):...
```

Load and process training and test sets

```
def train(batch_size=8, epochs=4, max_length=128):
```

- batch_size: the size of each training batch.
- epochs: the number of training rounds.

```
model = NewsClassifier(num_classes=5).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)
criterion = nn.CrossEntropyLoss()
```

Initialize model

- optimizer: uses the AdamW optimizer with a learning rate of 2e-5.
- criterion: use the cross-entropy loss function.

```
for epoch in range(epochs):
```

```
    for batch in progress_bar:...
```

```
        model.eval()
```

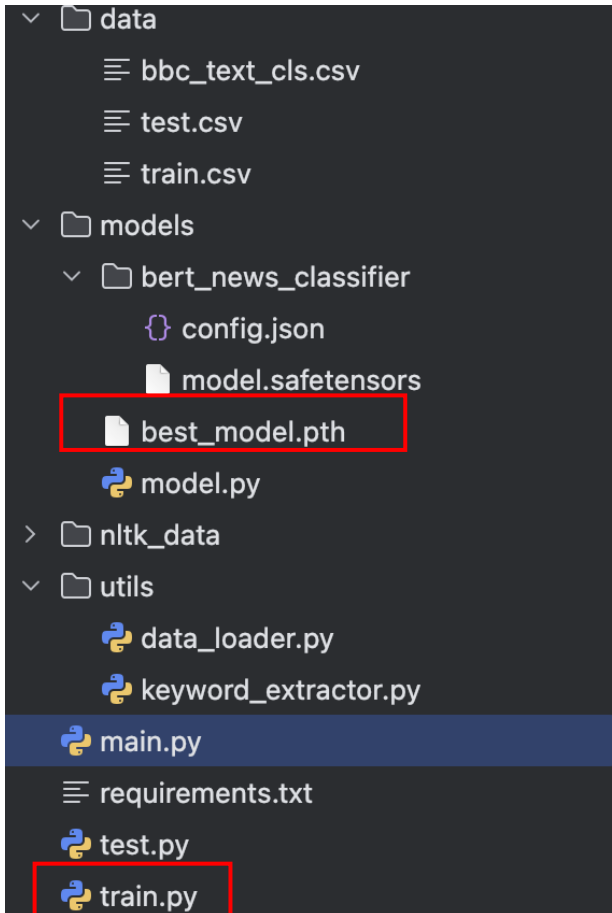
```
        correct = 0
```

```
        total = 0
```

```
        with torch.no_grad():...
```

- Training loop: the outer loop controls the number of rounds of training; the inner loop traverses the training data loader.
- Verify loop

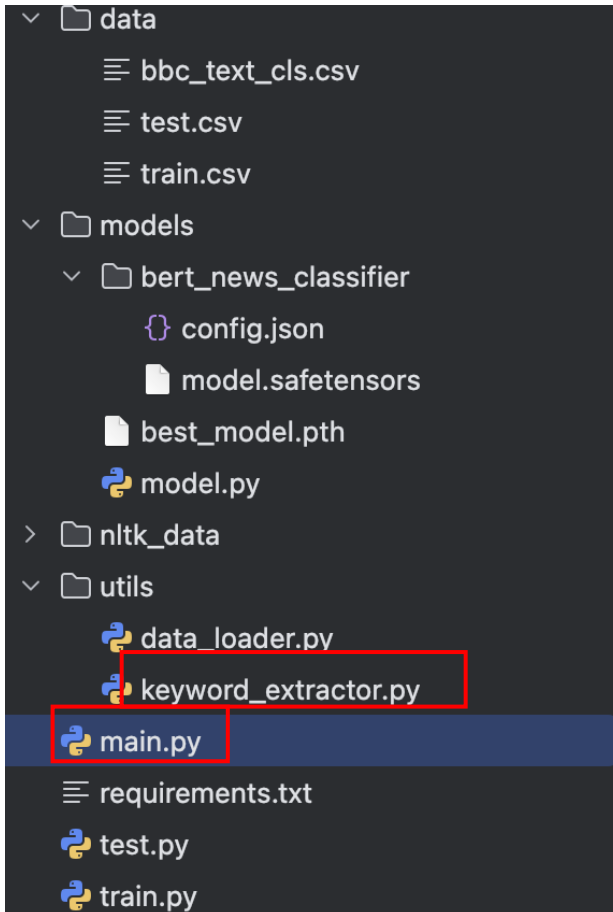
Save Trained Model



```
torch.save(obj: {  
    'model_state_dict': model.state_dict(),  
    'label_classes': train_dataset.label_encoder.classes_.tolist(), # Convert to Python list  
    'config': {  
        'max_length': max_length,  
        'model_type': 'distilbert' # Add model type identifier  
    }  
}, f: 'models/best_model.pth')
```

```
Epoch 1/4: 100%|██████████| 223/223 [00:47<00:00, 4.65it/s, loss=0.0121]  
Epoch 1 | Avg Loss: 0.3067 | Val Acc: 98.20%  
Epoch 2/4: 100%|██████████| 223/223 [00:47<00:00, 4.74it/s, loss=0.0789]  
Epoch 3/4: 0%|██████████| 0/223 [00:00<?, ?it/s]Epoch 2 | Avg Loss: 0.0567 | Val Acc: 98.43%  
Epoch 3/4: 100%|██████████| 223/223 [00:46<00:00, 4.75it/s, loss=0.00402]  
Epoch 4/4: 0%|██████████| 0/223 [00:00<?, ?it/s]Epoch 3 | Avg Loss: 0.0288 | Val Acc: 98.43%  
Epoch 4/4: 100%|██████████| 223/223 [00:46<00:00, 4.75it/s, loss=0.00136]  
Epoch 4 | Avg Loss: 0.0066 | Val Acc: 99.33%  
Model saved successfully
```

Chatbot



```
Initialize the device: mps
✓ Model checkpoint loaded successfully
/Users/Text Classifier/News_Classifier/.venv/lib/python3.12/site-packages/huggingface_hub/hf_api.py:100: FutureWarning: The `revision` argument is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a download, please use `force_download=True`.
  warnings.warn(
✓ Model architecture loaded successfully
Loading tag categories: ['business', 'entertainment', 'politics', 'sport', 'tech']
Text processor configuration complete (max_length=128)
🚀 Chatbot Initialisation Complete!
```

Please enter news content: *Alex Ovechkin scored his 894th goal, tying him with Wayne Gretzky in the NHL record books. In other news, the Pistons have shown remarkable improvement, climbing from the last position to a more competitive stance.*

Classification results: **SPORT** (99.2% 置信度)

Keywords: shown remarkable improvement, nhl record books, alex ovechkin scored

Please enter news content: *Stock markets experienced a sharp decline, with the Dow dropping 1,300 points, largely attributed to escalating trade tensions and tariffs implemented by the Trump administration.*

Classification results: **BUSINESS** (99.9% 置信度)

Keywords: stock markets experienced, escalating trade tensions, dow dropping 1

Please enter news content: *Lana Del Rey shared a rare insight into her life by posting pictures with her husband on tour. Additionally, celebrity Jinger Duggar responded to fans wanting to see more of her family.*

Classification results: **ENTERTAINMENT** (99.9% 置信度)

Keywords: lana del rey shared, celebrity jinger duggar responded, rare insight

Please enter news content: *A recent deal involving TikTok is reportedly on hold due to objections from China over tariffs. Meanwhile, advancements in AI are evident as Microsoft evaluates user engagement with its Copilot feature.*

Classification results: **TECH** (99.8% 置信度)

Keywords: recent deal involving tiktok, microsoft evaluates user engagement, hold due

Please enter news content: *The Trump administration is facing pressure after a significant market downturn linked to recent tariffs. Meanwhile, there are discussions to halt Biden's proposals concerning Medicare and Medicaid coverage.*

Classification results: **BUSINESS** (99.7% 置信度)

Keywords: significant market downturn linked, proposals concerning medicare, trump administration



RAG-Based Chatbot

RAG-Based Chatbot

Import News Files

```
def get_news_classification_from_files(files):
    news_classification_list = {}
    for file in files:
        try:
            if file.endswith(".csv"):
                try:
                    df = pd.read_csv(file, encoding='UTF-8') #如果不是utf-8或者latin1, 打不开csv
                except UnicodeDecodeError:
                    df = pd.read_csv(file, encoding='latin1')
                df_length = len(df['text'])
                news_texts = df['text']
                news_labels = df['label']
                for x in range(df_length): #将新闻正文和标签塞进字典里
                    text = news_texts[x]
                    label = news_labels[x]
                    if label in news_classification_list.keys():
                        news_classification_list[label].append(text)
                    else:
                        news_classification_list[label]=[text]

            if file.endswith(".txt"):
                try:
                    df = pd.read_csv(file, encoding='UTF-8')
                except UnicodeDecodeError:
                    df = pd.read_csv(file, encoding='latin1')
                df = pd.read_csv(file)
                df_readlines = df.readlines()
                text = df_readlines[1:] #txt的第一行是标签“商业, 娱乐”等, 第二行开始是正文
                label = df_readlines[0]
                if label in news_classification_list.keys():
                    news_classification_list[label].append(text) #将文档内容塞入list里面, 用于后续向量化
                else:
                    news_classification_list[label]=[text]

        except (FileNotFoundError, pd.errors.EmptyDataError, pymupdf.FileDataError) as e:
            print(f"Error during processing {file}: {e}")
    return news_classification_list
```

```
def get_news_keywords_from_files(files):
    news_keyword_extraction_list = {}
    for file in files:
        if file.endswith(".csv"):
            try:
                df = pd.read_csv(file, encoding='latin1')
            except UnicodeDecodeError:
                df = pd.read_csv(file, encoding='UTF-8')
            df_length = len(df['keywords'])
            news_keywords = df['keywords']
            news_texts = df['short_description']
            for x in range(df_length): #将新闻正文和标签塞进字典里
                text = news_texts[x]
                keyword = news_keywords[x]
                news_keyword_extraction_list[keyword] = text #将
    return news_keyword_extraction_list
```

Vectorized & Create Index

#3. 内容向量化

```
def convert_to_tfidf_vectors(vectorizer, docs): # 将新闻列表向量化
    texts = []
    for doc_list in docs.values():
        if isinstance(doc_list, list):
            texts.extend(doc_list) #extend the list of strings to the texts list.
        else:
            texts.append(doc_list) #if the value is a string, then append.
    texts_vectors = vectorizer.fit_transform(texts).toarray()
    return texts, texts_vectors

def create_faiss_index(texts_vectors): #创建一个《相似度搜索》的索引
    dimension = texts_vectors.shape[1]
    index = faiss.IndexFlatL2(dimension)
    index.add(texts_vectors.astype('float32'))
    return index

vectorizer = TfidfVectorizer(max_features=8000, min_df=5, max_df=0.5) #看看有无必要调低max_features
news_classification_list = get_news_classification_from_files(files_for_classification)
class_texts = []
```

RAG-Based Chatbot

Search Documents

```
def search_documents(vectorizer, index, query, top_k=3):    #利用索引搜索文本
    if not query:
        print("Empty query received.")
        return None, None
    query_vector = vectorizer.transform([query]).toarray().astype('float32')
    distances, indices = index.search(query_vector, top_k)
    return distances, indices

def indices_to_documents(distances, indices, texts, original_dict):    #索引到文档
    results = []
    for i in range(len(indices[0])):
        text = texts[indices[0][i]]
        distance = distances[0][i]
        similarity = 1 / (1 + distance)
        for key, value in original_dict.items():
            if isinstance(value, list) and text in value:
                label = key
                break
            elif isinstance(value, str) and text == value:
                label = key
                break
        results.append((label, text, similarity))
    return results
```

RAG-Based Chatbot

Run the LLM

```
def run():
    input_query = ""
    while input_query != "exit":
        input_query = input('Input a news for classification and keywords extraction: \n')
        if input_query != "exit":
            retrieved_knowledge_classification, retrieved_knowledge_keywords = retrieve(input_query)

            print('----- Retrieved Classification Knowledge -----')
            for label, text, similarity in retrieved_knowledge_classification:
                print(f'{label} similarity: {similarity:.2f}')

            print('----- Retrieved Keyword Extraction Knowledge -----')
            for keyword, text, similarity in retrieved_knowledge_keywords:
                print(f'{keyword} similarity: {similarity:.2f}')

            context_classification = '\n'.join([f' - (similarity: {similarity:.2f}) {label} \n\n {text}' for label, text, similarity in retrieved_knowledge_classification])
            context_keywords = '\n'.join([f' - (similarity: {similarity:.2f}) {keyword} \n\n {text}' for keyword, text, similarity in retrieved_knowledge_keywords])

            instruction_prompt = f"""
                You are a helpful chatbot for news classification and keywords extraction.
                Datasets for classification:
                {context_classification}
                Datasets for keyword extraction:
                {context_keywords}
                ---
                User's News:
                {input_query}
                ---
                Your mission is:
                1. News classification: Give the most similar Label of the news that user inputs. The output format is 'News Label:____'
                2. News keyword extraction: Analyze the User's News above and extract the most relevant keywords.
                These keywords MUST be specific to the User's News, not the datasets.
                The output format is 'News keywords:____'
                """

            messages = [{
                "role": "system",
                "content": instruction_prompt,
            }, {
                "role": "user",
                "content": input_query,
            }]

            tokenized_chat = chat_tokenizer.apply_chat_template(messages, tokenize=True, add_generation_prompt=True, return_tensors="pt", return_dict=True, return_attention_r
            print(f"prompt size: {len(tokenized_chat['input_ids'][0])}")
            generation_kwargs = tokenized_chat | {
                "pad_token_id": chat_tokenizer.eos_token_id,
                "max_new_tokens": 1000,
                "temperature": 0.2,
                "top_k": 1,
                "top_p": 0.8,
                "streamer": chat_streamer}
```

RAG-Based Chatbot Results

```
Downloading shards: 0%|          | 0/2 [00:00<?, ?it/s]
model-00001-of-00002.safetensors: 76%|#####5 | 3.75G/4.97G [00:00<?, ?B/s]
model-00002-of-00002.safetensors: 0%|          | 0.00/1.46G [00:00<?, ?B/s]
Loading checkpoint shards: 0%|          | 0/2 [00:00<?, ?it/s]
generation_config.json: 0%|          | 0.00/189 [00:00<?, ?B/s]
Input a news for classification and keywords extraction:
Donald Trump loves eating apples, while Joe Biden loves eating oranges.
----- Retrieved Classification Knowledge -----
entertainment similarity: 0.36
entertainment similarity: 0.36
tech similarity: 0.34
----- Retrieved Keyword Extraction Knowledge -----
sarah sanders flag anthem black and white similarity: 0.50
fox lake officer death black lives matter similarity: 0.50
petition to make princess leia an official disney princess similarity: 0.50
prompt size: 1427
News classification: entertainment

News keywords: trump, biden, apples, oranges

Input a news for classification and keywords extraction:
```

```
Input a news for classification and keywords extraction:
Coldplay will operate a concert in Hong Kong
----- Retrieved Classification Knowledge -----
sport similarity: 0.39
entertainment similarity: 0.39
business similarity: 0.39
----- Retrieved Keyword Extraction Knowledge -----
sarah sanders flag anthem black and white similarity: 0.50
fox lake officer death black lives matter similarity: 0.50
petition to make princess leia an official disney princess similarity: 0.50
prompt size: 1388
News classification: entertainment

News keywords: Coldplay, Hong Kong, concert

Input a news for classification and keywords extraction:
```

Prompt Engineering -Based Chatbot

Prompting Engineering Overview



This report will introduce three types of prompting methods: based, few-shot, and chain-of-thought (CoT).

These prompting techniques will be applied to a news classification task.

Data source:

We use 450 news test data, with feature sets of text and category

Base model (zero sample) Prompting

- **Basics of Prompt Engineering**

Directly providing the task description and input to the large language model, and asking the model to generate the corresponding output. For example, directly telling the model "This is a news article about sports, please classify it."

- **Basic Prompts**

Prompt:

```
"role": "system",  
"content": "You are a news classification expert. Classify news articles into: business,  
sport, tech, entertainment, politic. Respond ONLY with category:<predicted_category>"
```

- **Pros and Cons of Basic Prompts**

Pros: Simple and easy to use, no need for complex techniques.

Cons: The results may not be ideal, especially when dealing with complex news articles.

Few-shot Prompting

- **Few-Shot Prompt Principle**

Provide a few examples in the prompt, and tell the model how to classify them. For example, provide a few news articles that have already been classified, and let the model learn the pattern of classification.

- **Example of Few-Shot Prompts**

```
# Few-shot 示例
{
  "role": "user",
  "content": "Ad sales boost Time Warner profit\n\nQuarterly profits at US media giant"
},
{
  "role": "assistant",
  "content": "category:business"
},
{
  "role": "user",
  "content": "Gallery unveils interactive tree\n\nA Christmas tree that can receive te"
},
{
  "role": "assistant",
  "content": "category:tech"
},
```

- **Pros and Cons of Few-Shot Prompts**

Pros: Can improve the model's classification accuracy, especially when there is limited data.

Cons: Requires manual selection of appropriate examples, which can be costly.

Chain-of-Thought(CoT) Prompting

- **Chained Thinking Principle**

Guide the model to reason step-by-step, break down a complex task into multiple steps, and gradually generate the final answer, simulating the human thought process. This can enhance the model's ability to solve complex problems.

- **Example of Chained Thinking Prompts**

Prompt:

```
"role": "user",
"content": "Ad sales boost Time Warner profit\n\nQuarterly profits at US media giant rise as advertising sales surge"

"role": "assistant",
"content": ""Step-by-step analysis:
1. News content focuses on Time Warner's quarterly profit growth .
2. Key entities: advertising sales, financial reporting, company earnings .
3. Involves company's financial performance, which is in the business domain.
category:business""
```

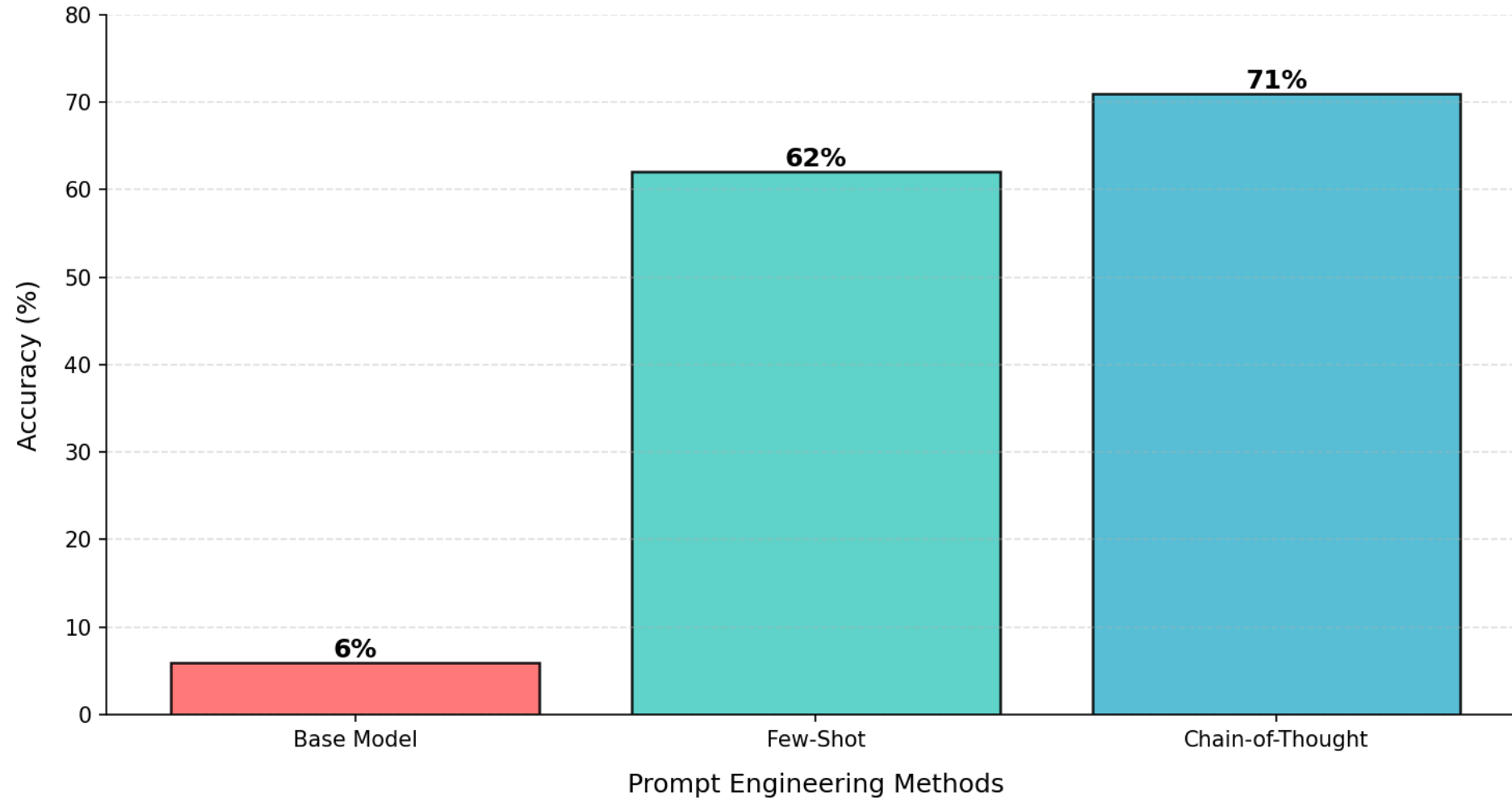
- **Pros and Cons of Chained Thinking**

Pros: Can handle complex news classification tasks and improve the model's reasoning ability.

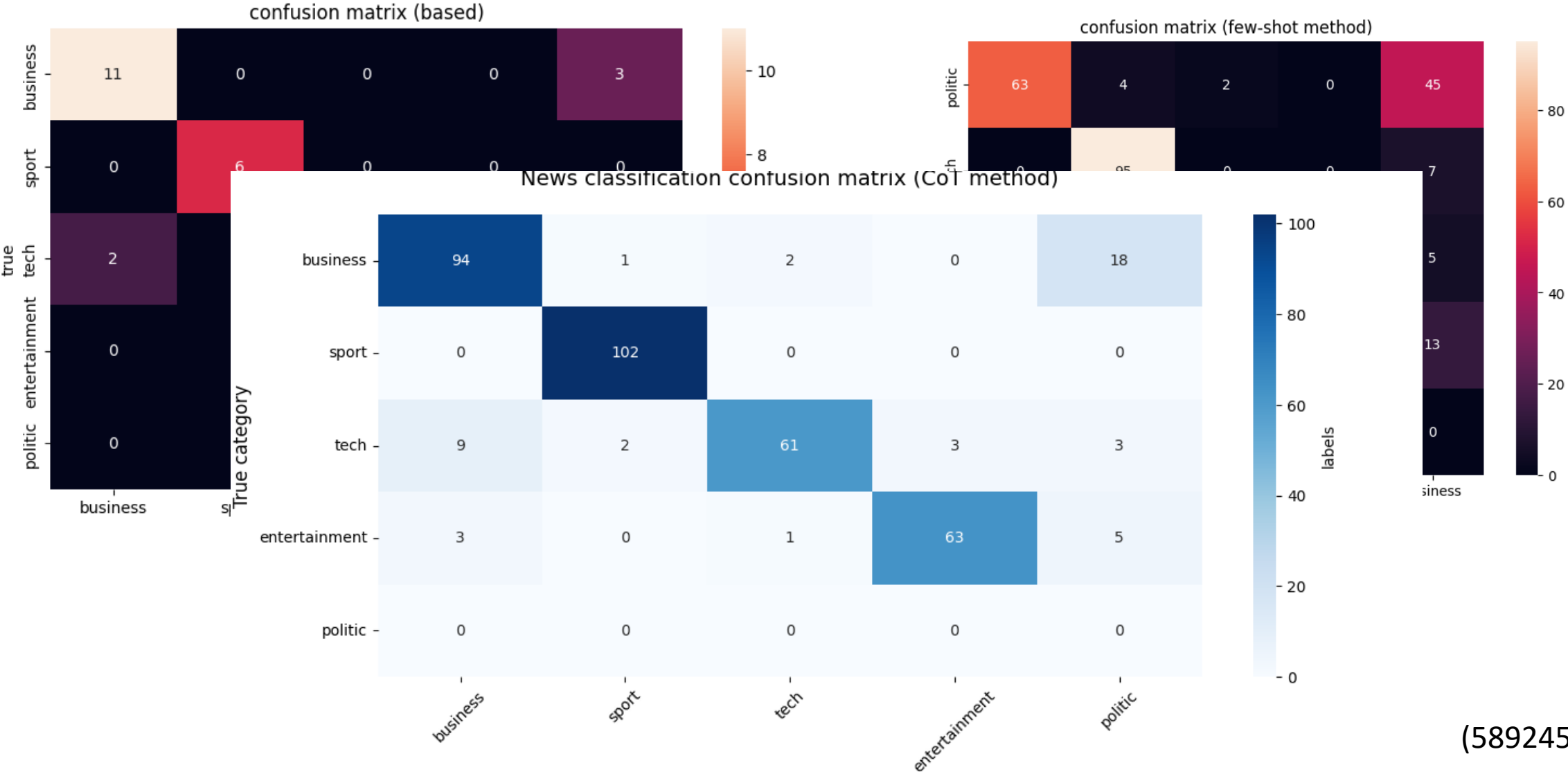
Cons: Requires designing a reasonable reasoning process, which can be challenging.

Comparison of experimental results

**Accuracy Comparison of Different Prompt Engineering Methods
(SCB10x-Llama3-1-Typhoon2-8B-Instruct)**



Comparison of experimental results



Problems and Challenge

Although the model has been fine-tuned with instructions, its zero-shot performance is still low (6%), and it relies on Few-Shot/CoT to improve results.

We can see that CoT shows significant advantages, increasing accuracy to 71% with step-by-step reasoning, but the accuracy is still not high.

- **To address these issues, we can:**
 - 1. **Few-Shot Optimization:** Match the most relevant context examples to the input questions.
 - 2. **CoT Refinement:** Design shorter or longer reasoning chains based on the model's capabilities to find the best balance. Add error-checking mechanisms (like having the model verify its reasoning steps).

Prompt Engineering -Based DistilBert&FLAN-T5



Prompt Experiment 2

model2: Distilbert-base-cased model

- fine-tuning
- prompt engineering

```
def train_evaluate(prompt_type="standard"):  
    # 初始化模型  
    tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-cased")  
    model = DistilBertForSequenceClassification.from_pretrained(  
        "distilbert-base-cased",  
        num_labels=len(labels),  
        id2label=id2label,  
        label2id=label2id  
    ).to(device)  
  
    prompter = ManualPromptGenerator(labels)  
  
    # 创建数据集  
    train_dataset = PromptDataset(X_train, y_train, tokenizer, prompter, prompt_type)  
    eval_dataset = PromptDataset(X_test, y_test, tokenizer, prompter, prompt_type)  
  
    # 训练配置  
    training_args = TrainingArguments(  
        output_dir=f"./results_{prompt_type}",  
        num_train_epochs=5,  
        per_device_train_batch_size=16,  
        per_device_eval_batch_size=32,  
        learning_rate=5e-5,  
        evaluation_strategy="epoch",  
        logging_steps=100,  
        disable_tqdm=True,  
        report_to="none"
```



learning-rate is 5e-5: Faster learning speed, suitable for simple classification tasks

epoch= 5: On small datasets, the model training parameters are able to converge



Prompt Engineering-Three prompt methods

- One-Shot

```
def _create_one_shot_examples(self):
    return {
        "business": {
            "text": "Tesla Q3 revenue reached $23.4B, up 9% YoY, with automotive gross margins at 18.9%.",
            "label": "business"
        },
        "tech": {
            "text": "Google announced its new TPU v5 chips with 3x faster AI training performance.",
            "label": "tech"
        },
        "politics": {
            "text": "Senate passes $95B foreign aid package with bipartisan 79-18 vote.",
            "label": "politics"
        },
        "sports": {
            "text": "NBA finals MVP scored 42 points in Game 7 to secure championship.",
            "label": "sports"
        },
        "entertainment": {
            "text": "Marvel's latest film grossed $185M in domestic opening weekend.",
            "label": "entertainment"
        }
    }
```

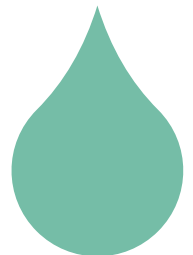
Under each category, manually add examples in text ->label structure.

- Few-Shot
- CoT



```
def _create_few_shot_examples(self):
    return [
        {
            "text1": "Amazon Web Services launched new AI inference chips",
            "text2": "Federal Reserve kept interest rates unchanged at 5.25-5.50%",
            "label": "business"
        },
        {
            "text1": "NASA's Mars rover found evidence of ancient microbial life",
            "text2": "Apple unveiled M3 chips with 24-core GPU",
            "label": "tech"
        },
        {
            "text1": "The upcoming midterm elections are set to be pivotal. This article explores key issues that voters care about",
            "text2": "Recent legislation aimed at climate change has sparked debate among lawmakers. Analysts weigh in on potential",
            "label": "politics"
        },
        {
            "text1": "The championship game featured an unexpected comeback, thrilling fans. This article breaks down the key plays",
            "text2": "A star athlete has announced their retirement, leaving a significant impact on the sport. This piece reflects",
            "label": "sports"
        },
        {
            "text1": "The latest superhero movie has shattered box office records. Critics analyze its cultural impact and what this means for the industry",
            "text2": "A popular streaming platform has announced a new series based on a bestselling novel. Fans are eager to see what happens next",
            "label": "entertainment"
        }
    ]
```

```
self.cot_steps = [
    "1. Identify key entities (companies, people, locations)",
    "2. Extract main topics and themes",
    "3. Analyze writing style and tone",
    "4. Determine most relevant category"
]
```



Dataset Distribution&Test Result Comparison

```
Using device: cuda
Detected 5 categories: ['business', 'entertainment', 'politics', 'sport', 'tech']

Dataset split: Train 1780, Test 445

=====
Strategy: STANDARD
=====

/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

tokenizer_config.json: 100% ██████████ 48.0/48.0 [00:00<00:00, 4.98kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 4.43MB/s]
```

```
{ 'loss': 0.0407, 'grad_norm': 0.04475109020835000, 'learning_rate': 2.3214285714285715e-05, 'epoch': 2.0704}
{'eval_loss': 0.07041523605585098, 'eval_accuracy': 0.9842696629213483, 'eval_runtime': 7.5783, 'eval_sample':
{'loss': 0.0173, 'grad_norm': 0.03829532861709595, 'learning_rate': 1.4285714285714285e-05, 'epoch': 3.5714}
{'eval_loss': 0.05971413478255272, 'eval_accuracy': 0.9887640449438202, 'eval_runtime': 7.9659, 'eval_sample':
{'loss': 0.0045, 'grad_norm': 0.027430091053247452, 'learning_rate': 5.357142857142857e-06, 'epoch': 4.4642}
{'eval_loss': 0.06294722110033035, 'eval_accuracy': 0.9887640449438202, 'eval_runtime': 7.9699, 'eval_sample':
{'train_runtime': 417.9763, 'train_samples_per_second': 21.293, 'train_steps_per_second': 1.34, 'train_loss':
{'eval_loss': 0.06294722110033035, 'eval_accuracy': 0.9887640449438202, 'eval_runtime': 7.4037, 'eval_sample':
```

1	0.4105	0.1222	0.9663
2	0.0743	0.0948	0.9798
3	0.0467	0.0704	0.9843
4	0.0173	0.0597	0.9888
5	0.0045	0.0629	0.9888

```
=== Final Results ===
| Strategy | Accuracy |
|-----|-----|
| STANDARD | 0.9865 |
| ONE SHOT | 0.9843 |
| FEW SHOT | 0.9820 |
| COT | 0.9888 |
```

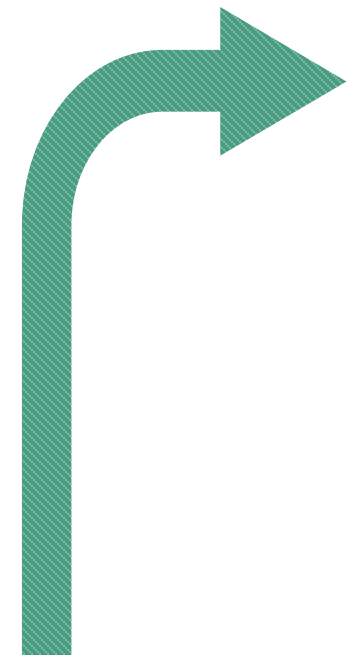


model3: Google-FLAN-T5-large model

```
# ===== Model Loading =====
model_name = "google/flan-t5-large"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name, device_map="auto", torch_dtype=torch.float16)
```

```
def _create_detailed_examples(self):
    """Create 2 examples per category"""
    return {
        "business": [
            {"text": "Tesla shares drop 15% after Q4 earnings miss expectations", "keywords": ["stock", "earnings", "company"]},
            {"text": "Federal Reserve announces 0.25% interest rate hike", "keywords": ["economy", "rate", "bank"]}
        ],
        "sport": [
            {"text": "Manchester United defeats Liverpool 2-1 in FA Cup final", "keywords": ["match", "goal", "player"]},
            {"text": "NBA All-Star Game breaks attendance record", "keywords": ["basketball", "game", "record"]}
        ],
        "tech": [
            {"text": "Google unveils Gemini AI model outperforming GPT-4", "keywords": ["AI", "model", "algorithm"]},
            {"text": "Apple announces M3 Ultra chip", "keywords": ["hardware", "chip", "performance"]}
        ],
        "politics": [
            {"text": "Senate passes $95B foreign aid bill", "keywords": ["bill", "senate", "law"]},
            {"text": "EU imposes new sanctions on Russia", "keywords": ["sanctions", "international", "trade"]}
        ],
        "entertainment": [
            {"text": "Oppenheimer wins Best Picture at Oscars 2024", "keywords": ["movie", "award", "actor"]},
            {"text": "Taylor Swift breaks Spotify record", "keywords": ["music", "album", "singer"]}
        ]
    }
```

- This model requires no training data, is fine-tuned by large-scale instructions, and is **sensitive** to natural language **prompts**.
- Add two typical examples under each of the five categories, in the nested dictionary structure with **category -> examples**. At the same time, **text -> keywords** is more specific and more structured



- Zero-Shot
- One-Shot

```
def zero_shot(self, text):
    return f"""Classify this news article into one category: {' , '.join(self.categories)}.
Article: {text}
Category: """
```

```
def one_shot(self, text):
    """One example per category"""
    examples = []
    for i, cat in enumerate(self.categories):
        example = self.examples[cat][0]
        examples.append(
            f"Example {i+1} ({cat}):\n"
            f"Article: {example['text']}\n"
            f"Keywords: {' , '.join(example['keywords'])}\n"
            f"Category: {cat}"
        )
    examples_str = "\n\n".join(examples)
    return f"""**Classification Task*: Match articles to categories based on keywords.

{examples_str}

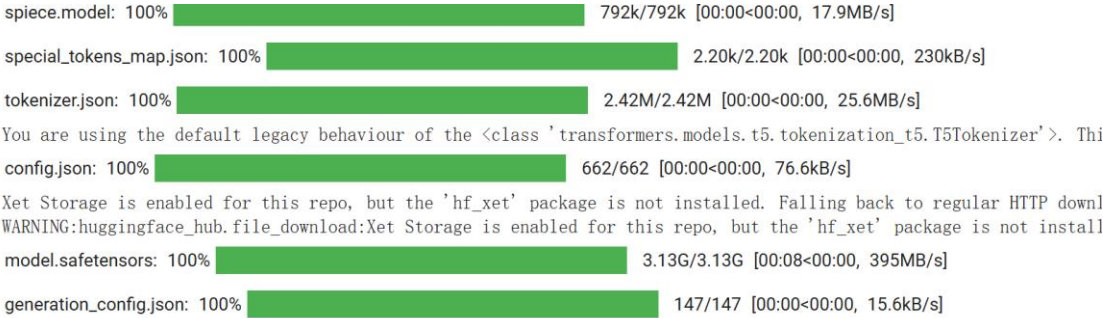
**New Article*:
{text}
Keywords: [Extract here]
Category: """
```

- Few-Shot
- CoT

```
def few_shot(self, text):
    """Two examples per category"""
    examples = []
    for cat in self.categories:
        for ex in self.examples[cat][:2]:
            examples.append(
                f"Article: {ex['text']}\n"
                f"Analysis: Keywords ({', '.join(ex['keywords'])}) -> {cat}"
            )
    examples_str = "\n\n".join(examples)
    return f"""**Classification Guidelines**:
- business: Company news, stock markets
- sport: Sports events, athletes
- tech: Technological innovations
- politics: Government decisions
- entertainment: Movies/TV, celebrity news
```

1. ****Content Summary**** (20 words max):
Summarize core content
2. ****Keyword Extraction****:
Identify 3-5 important terms
3. ****Category Matching****:
{clues_str}
4. ****Confidence Assessment****:
 - High: Matches ≥ 3 keywords
 - Medium: Matches 2 keywords
 - Low: Matches 1 keyword
5. ****Final Decision****:
Must select from [{', '.join(self.categories)}]

Data Distribution & Results



Data distribution:

label	
business	115
sport	102
tech	80
politics	76
entertainment	72

zero-shot

****Zero-Shot Results****

Prompt Example:

Classify this news article into one category: business, entertainment, politics, sport, tech

Article: UK house prices dip in November UK house prices dipped slightly in November

Category:

Classification Report:

	precision	recall	f1-score	support
business	0.90	0.94	0.92	115
entertainment	0.85	0.96	0.90	72
politics	0.95	0.88	0.91	72
sport	0.93	1.00	0.96	64
tech	1.00	0.84	0.91	80
accuracy			0.92	403
macro avg	0.93	0.92	0.92	403
weighted avg	0.93	0.92	0.92	403

one-shot

****New Article**:**

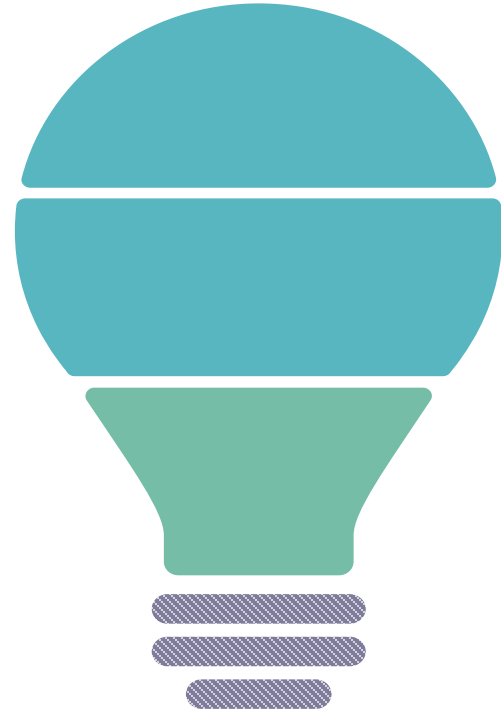
UK house prices dip in November UK house prices dipped slightly in November

Keywords: [Extract here]

Category:

Classification Report:

	precision	recall	f1-score	support
business	0.91	0.97	0.94	105
entertainment	0.88	0.98	0.92	44
politics	0.97	0.81	0.89	43
sport	0.98	1.00	0.99	58
tech	0.97	0.89	0.93	74
accuracy			0.94	324
macro avg	0.94	0.93	0.93	324
weighted avg	0.94	0.94	0.94	324



Results & Conclusion

Few-shot

****New Article**:**

UK house prices dip in November UK house prices dipped slightly

Analysis: Keywords ([Extract keywords]) -> Category:

Classification Report:

	precision	recall	f1-score	support
business	0.94	0.90	0.92	113
entertainment	0.97	0.92	0.94	65
politics	0.88	0.93	0.90	75
sport	0.94	1.00	0.97	102
tech	0.99	0.94	0.96	78
accuracy			0.94	433
macro avg	0.94	0.94	0.94	433
weighted avg	0.94	0.94	0.94	433

CoT

Step-by-Step Analysis:

1. Summary:

Classification Report:

	precision	recall	f1-score	support
business	0.71	1.00	0.83	5
entertainment	1.00	0.33	0.50	3
politics	1.00	1.00	1.00	3
sport	1.00	1.00	1.00	1
tech	1.00	1.00	1.00	2
accuracy			0.86	14
macro avg	0.94	0.87	0.87	14
weighted avg	0.90	0.86	0.83	14



Final Performance Comparison

Strategy	Accuracy	Coverage
Zero-Shot	0.921	403/445
One-Shot	0.938	324/445
Few-Shot	0.940	433/445
CoT	0.857	14/445

- The performance of **Few-shot** is the best, and CoT method is more suitable for mathematical derivation and complex logical reasoning classification.
- In simple text topic classification tasks, complex stepwise reasoning requirements (e.g., "keyword extraction → matching category → confidence evaluation") cause the model to refuse to predict most samples, which reduces the model ability.

Benchmark

Benchmark

- Our goal is to **test the accuracy** of Fine-tuning and RAG models based on **real news** and try to **avoid overfitting** problem and **uncover biases**
- Divide the dataset into two categories to evaluate the performance of our LLM models **under simple and complex text environment**
- Compare the model outputs with the original labels to observe whether the model **meets a certain standard of reliability** before deployment

Text	Label	Class
NovakDjokovicWinsRecord25thGrandSlamatWimble don2024 DjokovicdefeatedCarlosAlcarazinahistoricfive- setmatch,solidifyinghisstatusasthemostdecoratedtenni splayer.	sport	true
OlympicArcherDisqualifiedforUsingLaser- GuidedBow FrencharcherJeanDupontwasexpelledafterofficialsdisc overedillegaldevicesinhisequipment.	sport	true
TeslaReportsRecordQuarterlyRevenueof\$25Billion Tesla'sQ3earningsshowa12%revenueincreasedrivenby strongModelYsales.Sharesrose5%inafter- hourstrading.	business	true
...	...	...

Result

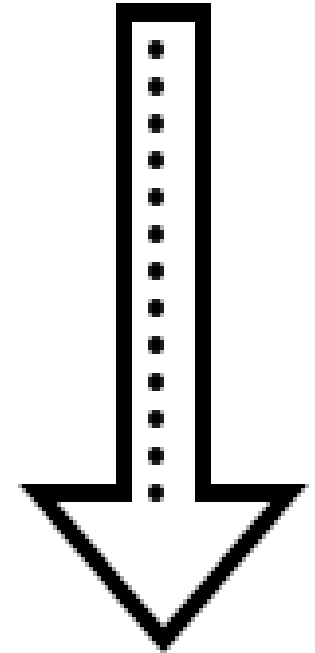
- The performance of Fine-tuning model is **relatively better** than RAG's performance
- Fine-tuning model identifies and classifies **Entertainment, Sport and Business news** more accurately, while RAG model has more advantage over **Entertainment, Tech and Politics news**
- The number of times of Fine-tuning model mistakenly identifying politics news as business news is the most

	Accuracy(Fine-tuning)	Accuracy(RAG)
Overall	70.00%	50.00%
Simple	90.00%	70.00%
Complex	50.00%	30.00%

	Accracy(Both)	Accracy(Fine-tuning)	Accracy(RAG)
Entertainment	75.00%	100.00%	75.00%
Tech	50.00%	50.00%	50.00%
Sport	40.00%	100.00%	40.00%
Business	33.33%	66.67%	33.33%
Politics	25.00%	25.00%	50.00%

Limitation Analysis

- The **limited size of training dataset and testing dataset**, as well as the insufficient capacity and functionality of the original LLM model
- The **ambiguous and misleading discription** of the content of news, as well as the universality and **multi-label** of news
- The **lack of explicit feature awareness** and ability to memorize training dataset instead of learning general patterns



Future Work

- Use larger, balanced datasets with clear labels and specific LLM models for training
- Apply techniques like **dropout or label smoothing** to reduce overfitting and **add task-specific layers** to the base models
- Improve the retriever with **dense vector search** and **filter retrieved documents for relevance** before feeding to the generator
- Test robustness with **adversarial examples and ambiguous headlines**



Thanks

Q & A